

Dynamic Programming (III)

- Longest common subsequence
- From Traceback to Backtracking
 - Eight Queens

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

Longest Common Subsequence (LCS)

- A subsequence of a sequence/string S is obtained by deleting zero or more symbols from S .
- For example, the following are **some** subsequences of “president”: pred, sdn, predent. In other words, the letters of a subsequence of S appear in order in S , but they are not required to be consecutive.
- The longest common subsequence problem
 - Given two strings **X** and **Y** with length **m** and **n**
 - find a maximum length common subsequence between them.

Longest Common Subsequence

- Subsequence:
 - BDFH is a **subsequence** of ABCDEFGH
- If X and Y are sequences, a **common subsequence** is a sequence which is a subsequence of both.
 - BDFH is a **common subsequence** of ABCDEFGH and of ABDFGHI
- A **longest common subsequence**...
 - ...is a common subsequence that is longest.
 - The **longest common subsequence** of ABCDEFGH and ABDFGHI is ABDFGH.

- Applications in **bioinformatics**



- The unix command **diff**
- Merging in version control
 - **svn**, **git**, etc...

```
anari — anari@nimbook — ...
~ cat file1
A
B
C
D
E
F
G
H
~ cat file2
A
B
D
F
G
H
~ diff file1 file2
3d2
< C
5d3
< E
8a7
> I
```

Recipe for applying Dynamic Programming

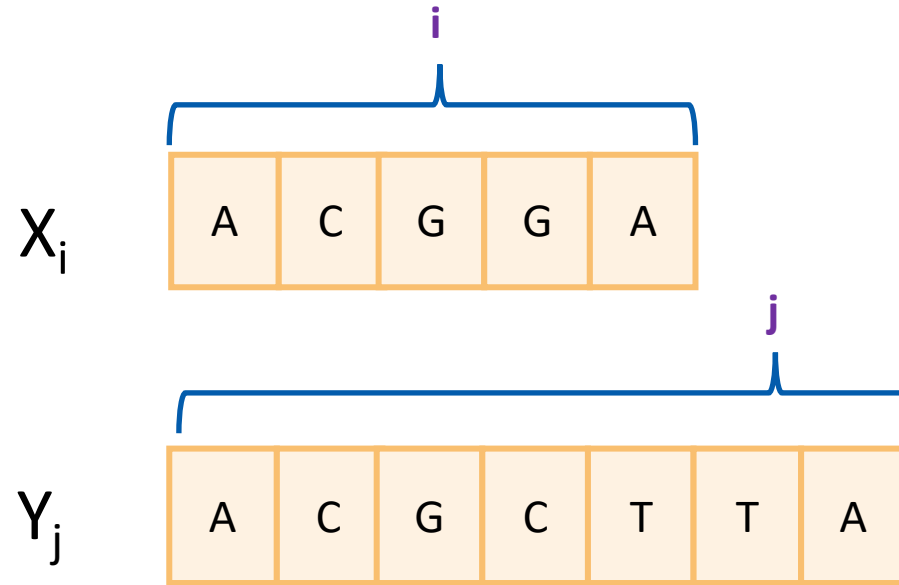
- **Step 1:** Characterize the structure of an optimal solution. 
- **Step 2:** Define the optimal value recursively.
- **Step 3:** Compute the optimal values using dynamic programming.
- **Step 4:** If needed, keep track of some additional info so that the algorithm from Step 3 can find the actual LCS



- Examples: $C[2,3] = 2$
 $C[4,4] = 3$

Step 1: Identify the Optimal Substructure

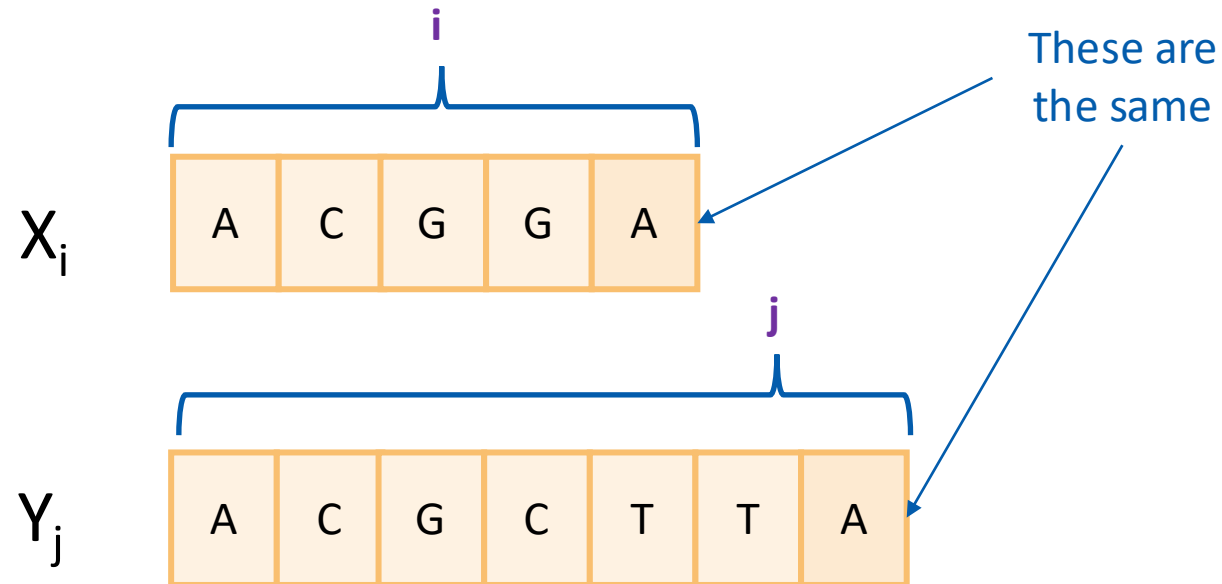
- Write $C[i,j]$ in terms of the solutions to smaller sub-problems



$$C[i,j] = \text{length_of_LCS}(X_i, Y_j)$$

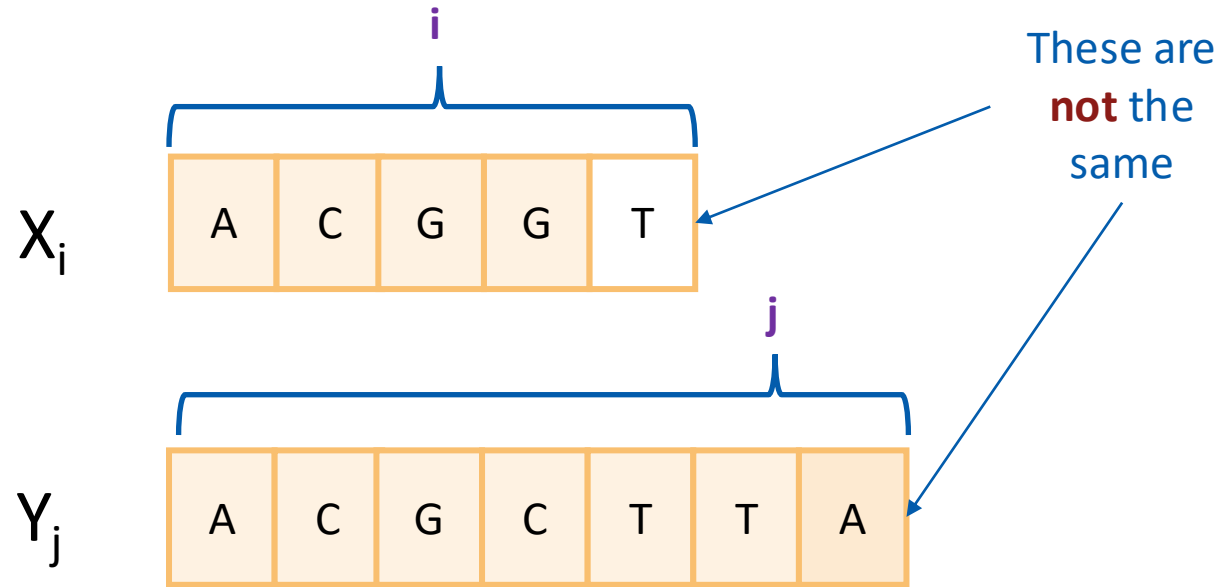
Two cases

Case 1: $X[i] = Y[j]$



- Then $C[i,j] = 1 + C[i-1,j-1]$. **Why?**
 - An LCS can use the matching last symbols as its final symbol.
 - Removing this symbol must leave an LCS of X_{i-1} and Y_{j-1} .
 - Otherwise, a longer solution for those prefixes would give a longer common subsequence of X_i and Y_j .

Case 2: $X[i] \neq Y[j]$

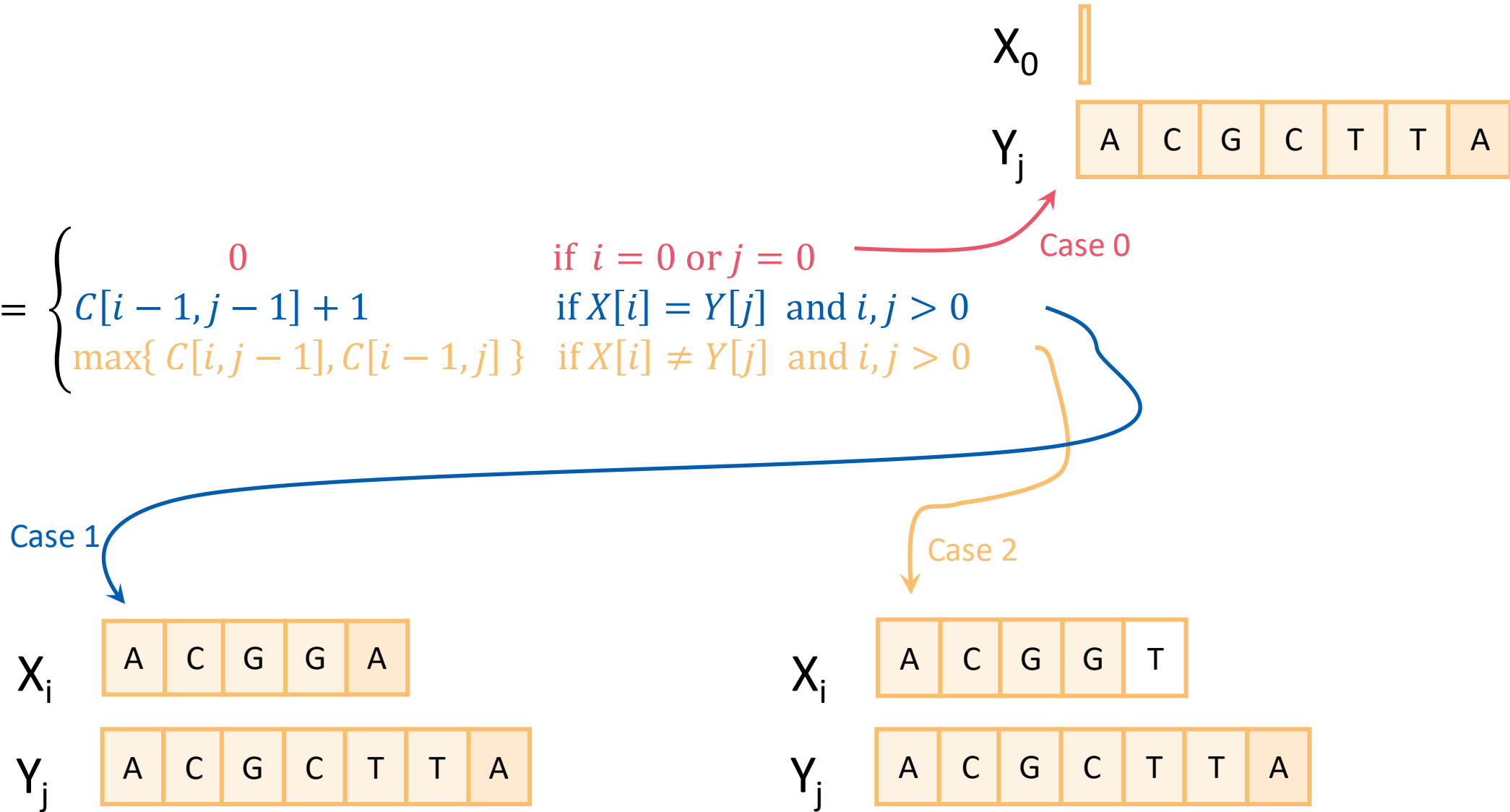


- Then $C[i,j] = \max\{ C[i-1,j], C[i,j-1] \}$.
 - either $LCS(X_i, Y_j) = LCS(X_{i-1}, Y_j)$ and **T** is not involved,
 - or $LCS(X_i, Y_j) = LCS(X_i, Y_{j-1})$ and **A** is not involved,

Why: A common subsequence cannot use both final positions!

Recurrence for the LCS Length

$$\bullet C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i - 1, j - 1] + 1 & \text{if } X[i] = Y[j] \text{ and } i, j > 0 \\ \max\{C[i, j - 1], C[i - 1, j]\} & \text{if } X[i] \neq Y[j] \text{ and } i, j > 0 \end{cases}$$



- Direct recursion may solve the same subproblem many times.
- For example, computing $C[i-1,j]$ and $C[i,j-1]$ may both require $C[i-1,j-1]$.
- There are only $(m+1)(n+1)$ different subproblems.
- We compute each answer once and store it in a table.

- **LCS(X, Y):**
 - $C[i,0] = C[0,j] = 0$ for all $i = 0, \dots, m, j=0, \dots, n$.
 - **For** $i = 1, \dots, m$ and $j = 1, \dots, n$:
 - **If** $X[i] = Y[j]$:
 - $C[i,j] = C[i-1,j-1] + 1$
 - **Else:**
 - $C[i,j] = \max\{ C[i,j-1], C[i-1,j] \}$
 - **Return** $C[m,n]$

*Running time:
 $O(nm)$*

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{ C[i,j-1], C[i-1,j] \} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Example

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

A C T G

X
A
C
G
G
A

0	0	0	0	0
0				
0				
0				
0				
0				

X
A C G G A

Y
A C T G

Example

		Y				
		A	C	T	G	
X	A	0	0	0	0	0
	C	0	1	1	1	1
	G	0	1	2	2	2
	G	0	1	2	2	3
	A	0	1	2	2	3

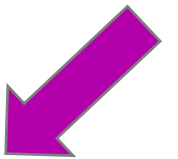
$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i - 1, j - 1] + 1 & \text{if } X[i] = Y[j] \text{ and } i, j > 0 \\ \max\{C[i, j - 1], C[i - 1, j]\} & \text{if } X[i] \neq Y[j] \text{ and } i, j > 0 \end{cases}$$

X	A	C	G	G	A
Y	A	C	T	G	

So the LCS of X and Y has length 3.

Recipe for applying Dynamic Programming

- **Step 1:** Characterize the structure of an optimal solution.
- **Step 2:** Define the optimal value recursively.
- **Step 3:** Compute the optimal values using dynamic programming.
- **Step 4:** If needed, keep track of some additional info so that the algorithm from Step 3 can find the actual LCS



Recovering an LCS

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

		0	0	0	0	0
	A	0	1	1	1	1
	C	0	1	2	2	2
X	G	0	1	2	2	3
	G	0	1	2	2	3
	A	0	1	2	2	3

X	A	C	G	G	A
---	---	---	---	---	---

Y	A	C	T	G
---	---	---	---	---

Algorithm:

1. Start at $(i,j) = (m,n)$.
2. If $X[i] = Y[j]$:
Record the matching symbol and move diagonally.
3. Otherwise:
Move up or left, choosing the larger table entry.
If the entries are equal, choose either.
4. Stop when $i = 0$ or $j = 0$.
5. Reverse the recorded symbols to obtain an LCS.

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X

A
C
G
G
A

0	0	0	0	0
0	1	1	1	1
0	1	2	2	2
0	1	2	2	3
0	1	2	2	3
0	1	2	2	3

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- Start at C[m,n].

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

X

A
C
G
G
A

0	0	0	0	0
0	1	1	1	1
0	1	2	2	2
0	1	2	2	3
0	1	2	2	3

That 3 must have come from the 3 above it.

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X

A
C
G
G
A

0	0	0	0	0
0	1	1	1	1
0	1	2	2	2
0	1	2	2	3
0	1	2	2	3
0	1	2	2	3

This 3 came from that 2 –
we found a match!

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- The symbols match.
- Record G and move diagonally.

G

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X

A
C
G
G
A

0	0	0	0	0
0	1	1	1	1
0	1	2	2	2
0	1	2	2	3
0	1	2	2	3
0	1	2	2	3

That 2 may as well
have come from
this other 2.

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- The upper and left entries have the same value.
- Both choices preserve the optimal length.
- To recover one LCS, choose either direction.

G

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

		0	0	0	0	0
	A	0	1	1	1	1
	C	0	1	2	2	2
X	G	0	1	2	2	3
	G	0	1	2	2	3
	A	0	1	2	2	3

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- A diagonal jump means that we found an element of the LCS!

G

Example – traceback

$$C[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1,j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i,j > 0 \\ \max\{C[i,j-1], C[i-1,j]\} & \text{if } X[i] \neq Y[j] \text{ and } i,j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X

A
C
G
G
A

0	0	0	0	0
0	1	1	1	1
0	1	2	2	2
0	1	2	2	3
0	1	2	2	3

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- A diagonal jump means that we found an element of the LCS!
- Stop when $i = 0$ or $j = 0$

C	G
---	---

Example – traceback

$$C[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ C[i-1, j-1] + 1 & \text{if } X[i] = Y[j] \text{ and } i, j > 0 \\ \max\{C[i, j-1], C[i-1, j]\} & \text{if } X[i] \neq Y[j] \text{ and } i, j > 0 \end{cases}$$

Y

A	C	T	G
---	---	---	---

X		0	0	0	0
	A	0	1	1	1
	C	0	1	2	2
	G	0	1	2	3
	G	0	1	2	3
	A	0	1	2	3

X

A	C	G	G	A
---	---	---	---	---

Y

A	C	T	G
---	---	---	---

- We record the symbols in reverse order: G, C, A.
- Reversing them gives an LCS:

A	C	G
---	---	---

This process to find an LCA is called **traceback**

- we follow the recurrence backwards to recover an optimal solution.

From One LCS to All LCSs

		Y			
		B	A	C	
		0	0	0	0
A	0	0	1	1	
B	0	1	1	1	
C	0	1	1	2	

At $C[2,2]$, both up and left still preserve the optimal value 1.

$X = ABC, Y = BAC$

		AC		
X	A	B	C	
Y	B	A	C	

		BC		
X	A	B	C	
Y	B	A	C	

Both are optimal LCSs of length 2. The previous slide's single walk no longer tells the whole story.

A single traceback recovers one LCS.

Question: What if we want all distinct LCSs?

To Find all LCSs, A single walk is not enough

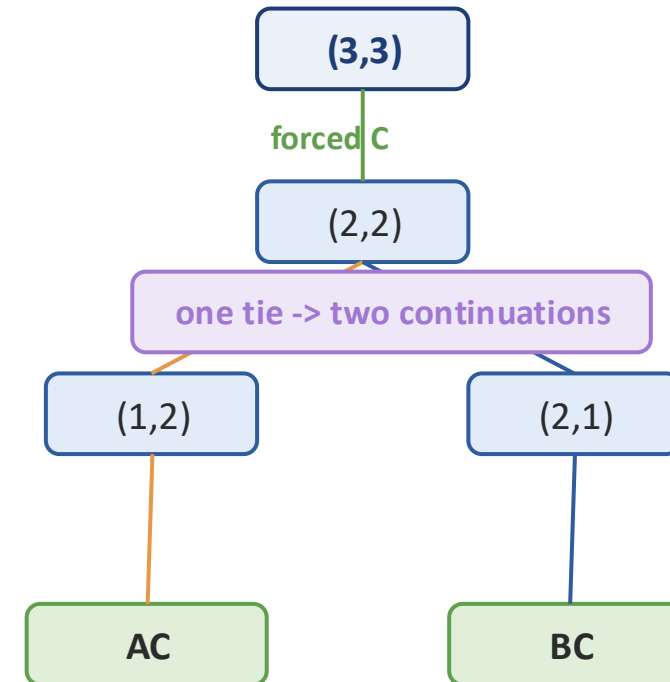
What changes at a tie?

match -> go diagonal and append the symbol

strictly better side -> follow it

tie -> now there are two equally good next states

To list every optimal LCS, we must recursively explore both branches at each tie



What is Backtracking?

Formal definition

Backtracking is depth-first search on a tree of partial solutions.

- We try a choice and explore the resulting branch recursively.
- When that branch fails or has been fully explored, we return to the previous choice point and try another choice.

Key ingredients: partial solution, choice point, recurse, return.

The return step is the “backtracking” part.

choose



explore



return

then try others

What have we learned?

- A single trace-back recovers one optimal LCS.
- When ties appear, the traceback becomes a search tree
 - We explore both branches
- **Backtracking = choose, recurse, return, then try other branches.**
- Next: 8-Queens uses the same search idea without a DP table.

Eight Queens Problem

Place 8 queens on an 8 x 8 chessboard:

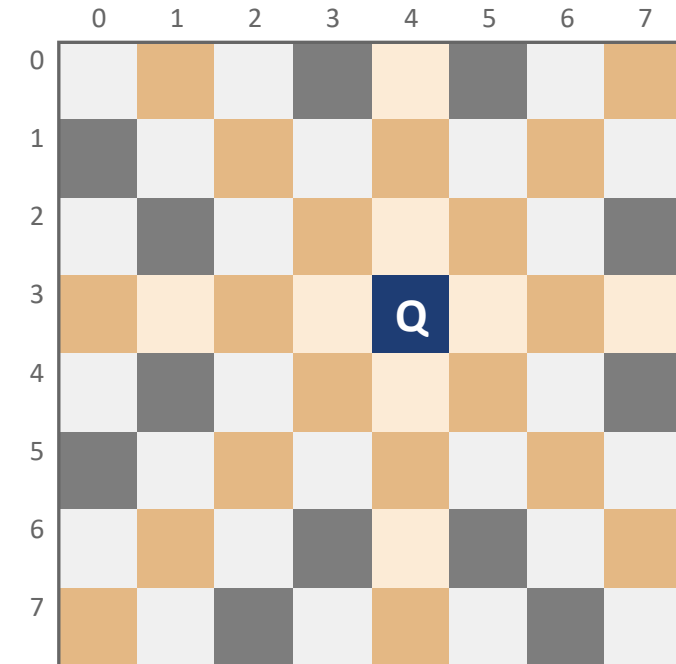
- So that no two queens share a row, a column, or a diagonal.
- **Goal:** find all valid placements

Equivalent search state:

- How to place one queen (choose one column) for each row

state[row] = chosen column

Once the row is fixed, only the column is a decision.



One queen attacks along its row, column, and both diagonals.

Example Solution

	0	1	2	3	4	5	6	7
0	Q							
1					Q			
2								Q
3						Q		
4			Q					
5							Q	
6		Q						
7				Q				

One valid placement

$Q = [0, 4, 7, 5, 2, 6, 1, 3]$

Entry i stores the column chosen for row i .

Every row is filled exactly once.

Backtracking builds this array from left to right.

Backtracking in LCS and 8-Queens

LCS: enumerate all optimal answers

Partial solution: Current cell (i,j) and the symbols recorded so far.

Choices: the three cells allowed by the LCS recurrence.

Pruning: Discard transitions inconsistent with the recurrence.

Output: Each distinct LCS, with duplicates removed.

8-Queens: enumerate all valid placements

Partial solution: The queens placed in earlier rows.

Choices: Columns in the current row.

Pruning: Reject positions attacked by an earlier queen.

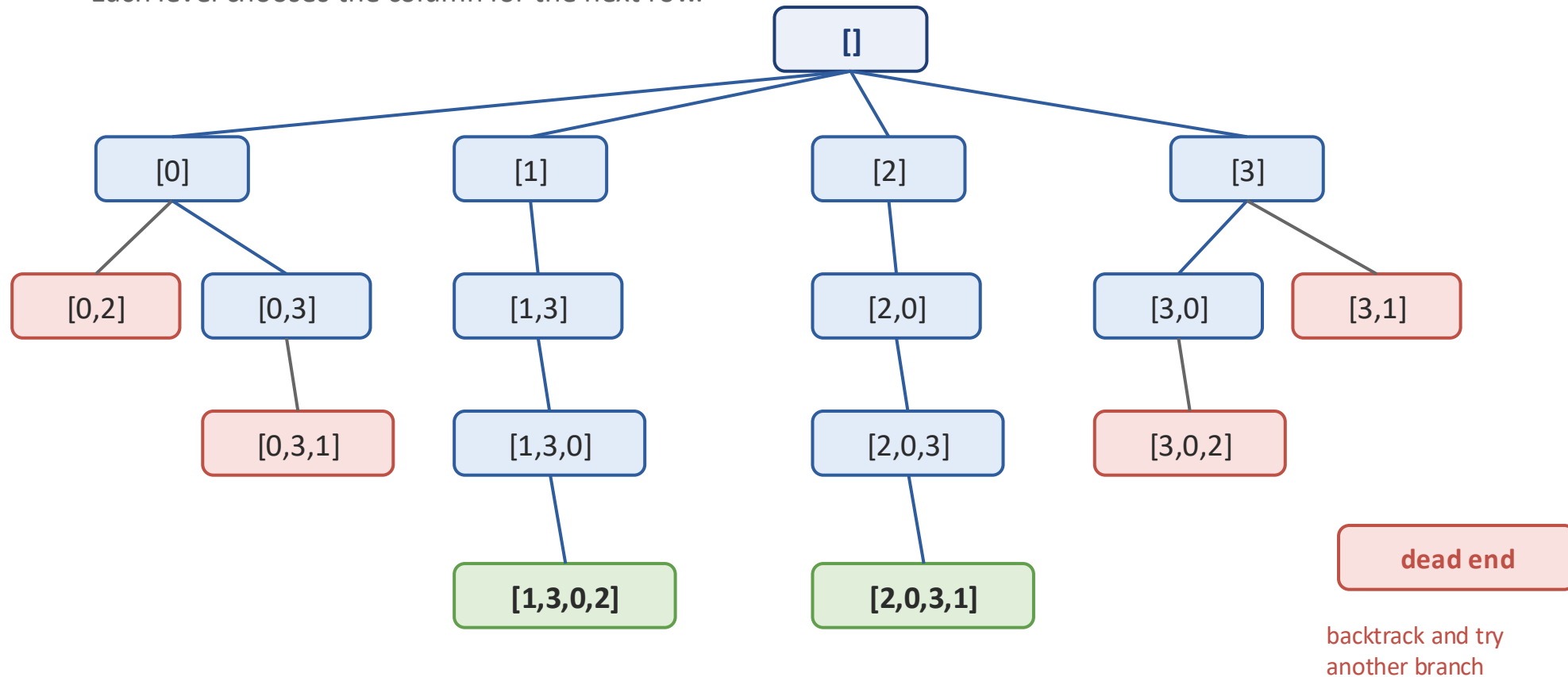
Output: Each complete valid placement.

Same search pattern, different pruning rules.

Backtracking Search Tree

illustrated for $n = 4$

Each level chooses the column for the next row.



Backtracking Idea:

- We place queens row by row, so recursion depth is the current row.
- A node in the search tree is just a partial array `pos[0..row-1]`.
- When testing `(row, col)`, we only compare against earlier rows
- The moment a partial placement conflicts, we prune that branch.

Remaining question:

- How to check if a position `(row, col)` is safe?

N-Queens Algorithm

```
Backtrack(row):  
    if row == n:  
        output solution  
        return  
  
    for col in 0 .. n-1:  
        if safe(row, col):  
            place queen  
            mark column and diagonals  
            Backtrack(row + 1)  
            unmark and remove queen
```

1. try a safe column

2. recurse to next row

3. undo on return

The undo step is what makes the search backtrack cleanly.

Checking Safety in $O(1)$ time

Idea:

- maintain all occupied columns and diagonals using hashing
- Each test becomes a few $O(1)$ hash lookups
- Observation: we can assign each diagonal an ID such that
 - for every entry at (row, col), its two diagonal IDs are $\text{col} - \text{row}$ and $\text{row} + \text{col}$

Diagonal ID = $\text{col} - \text{row}$

0	1	2	3	
-1	(0,0)	(0,1)	(0,2)	(0,3)
-2	(1,0)	(1,1)	(1,2)	(1,3)
-3	(2,0)	(2,1)	(2,2)	(2,3)
	(3,0)	(3,1)	(3,2)	(3,3)

Diagonal ID = $\text{row} + \text{col}$

	0	1	2	3	
(0,0)	(0,1)	(0,2)	(0,3)	4	
(1,0)	(1,1)	(1,2)	(1,3)	5	
(2,0)	(2,1)	(2,2)	(2,3)	6	
(3,0)	(3,1)	(3,2)	(3,3)		

Checking Safety in $O(1)$ time

- Each test becomes a few $O(1)$ hash lookups.
- A position is safe if its column and both diagonals are unoccupied
- Maintain the following three hash tables

usedCol[col]

usedDiag1[col - row]

usedDiag2[row + col]

Example on an 8 x 8 board

try (row, col) = (3, 1)

usedCol[1]

usedDiag1[1-3] = usedDiag1[-2]

usedDiag2[1+3] = usedDiag2[4]

With a dict/set, negative keys are fine.

The search tree has at most $O(n!)$ nodes.

Each call tests at most n columns.

With hash-set safety checks, the running time is $O(n \cdot n!)$ in expectation.

Backtracking cuts off bad partial placements immediately.

Space is $O(n)$: recursion depth n plus three bookkeeping arrays.

**Time
exponential**

**Space
 $O(n)$**

Stopping after the first solution is typically much faster than listing every solution.